

Tema 4: Desarrollo Basado en Componentes con React

1. Desarrollo Basado en Componentes

El paradigma de desarrollo basado en componentes consiste en dividir la interfaz de usuario (IU) en partes más pequeñas e independientes. Cada componente es responsable de generar una porción de la IU y de responder a los eventos de usuario relativos a esa parte.

- **Ventajas:** Facilita la estrategia de "divide y vencerás" para construir interfaces complejas mediante elementos pequeños, y promueve la **reusabilidad** del código (tanto dentro de una misma aplicación como entre varias).
 - **Ejemplos:** Este paradigma es el estándar en los frameworks para aplicaciones SPA (*Single Page Application*) modernos, tales como React, Angular y Vue.js.
-

2. Introducción a React y Sintaxis JSX

React es la librería que empleamos para construir nuestras interfaces. Utiliza **JSX**, una extensión de sintaxis que permite escribir código con apariencia de HTML directamente dentro de los archivos JavaScript.

2.1. Reglas de Examen sobre JSX

Las preguntas sobre la sintaxis de JSX son un clásico en los exámenes. Presten especial atención a lo siguiente:

- **Nombres de atributos:** Dentro de una expresión JSX, **NO** todos los atributos de las etiquetas HTML se pueden usar con su nombre original. El caso más importante es el atributo `class` de HTML, que en JSX debe escribirse obligatoriamente como `className`.
 - **Variables en JSX:** Para incrustar el valor de una variable JavaScript dentro de una expresión JSX, se deben utilizar las llaves `{variable}`. **Nunca** se debe usar la sintaxis `@variable`.
 - **Transformación previa:** El navegador web no entiende los ficheros JSX de forma nativa; requiere una transformación previa para convertirse en código JavaScript estándar. En nuestro proyecto, **Vite** es la herramienta de construcción ultrarrápida encargada de realizar esta transformación por debajo.
-

3. Estado Local de un Componente (`useState`)

Los componentes funcionales de React gestionan su estado interno mediante el uso de *Hooks*, siendo `useState` el principal de ellos.

Ejemplo de Examen:

Analicemos el siguiente fragmento de código extraído de pruebas de evaluación anteriores:

```
const App = () => {
  const [todos, setTodos] = useState([]);
  const handleAddTodo = text => setTodos([todo(text), ... todos]);
  return (
    <div>
      <AddTodo onAddTodo={handleAddTodo}/>
      <Todos todos={todos}/>
    </div>
  );
}
```

- **Regla de Inicialización:** La instrucción `useState([])` inicializa la variable `todos` como un array vacío únicamente en la **primera renderización**. En las renderizaciones sucesivas, `todos` conserva su valor actual; no vuelve a inicializarse a `[]`.
- **Regla de Re-renderizado:** La ejecución de `handleAddTodo` (que internamente llama a `setTodos` modificando el estado) causará que el componente se vuelva a renderizar para reflejar los cambios en pantalla.
- **Alcance del estado:** El valor de `todos` en este ejemplo es estrictamente local al componente `App`. No se guarda automáticamente en un estado global (como Redux).

4. El Virtual DOM

El Virtual DOM es el mecanismo central que permite a React ser rápido y eficiente.

- **Regla de Examen:** Los componentes React se renderizan en el Virtual DOM.
- **Funcionamiento:** React mantiene en memoria una copia ligera de la estructura de la interfaz (Virtual DOM). Cuando el estado de un componente cambia, React genera un nuevo árbol del Virtual DOM, lo compara con la versión anterior (proceso conocido como *diffing*), y aplica en el DOM real del navegador solo las diferencias encontradas, evitando costosas recargas de página o repintados innecesarios.

5. Inmutabilidad

Para que el Virtual DOM y sus mecanismos de optimización funcionen de manera predecible, los datos en React (tanto el estado local como las propiedades o *props*) deben tratarse como **inmutables**.

- **Igualdad referencial:** React realiza un test de igualdad rápido llamado *shallow comparison* (comparación superficial o referencial) para determinar si debe re-renderizar un componente.
- **Creación de nuevos objetos:** Si modificamos un array directamente (por ejemplo, haciendo `todos.push(nuevo)`), la referencia en memoria del array sigue siendo la misma, por lo que React podría no detectar el cambio. Es obligatorio crear copias nuevas. Por eso, en el código anterior, se genera un array completamente nuevo: `setTodos([todo(text), ...todos])`.
- **Beneficios adicionales:** Trabajar con estructuras inmutables no solo mejora la eficiencia del renderizado, sino que también facilita la implementación de funcionalidades complejas como deshacer/rehacer (*undo/redo*).